

Operational Metadata as a Property Graph: A Reference Model and Query Workload for Lakehouse Reliability

Mogana Kumaran Sivaraman

San Francisco, CA, USA.

Corresponding author(s). E-mail(s): moganakumaran@ieee.org;

Abstract

Reliability engineering on a data lakehouse asks questions that are graph traversals: which consumers a schema change breaks, why a table is stale, which upstream change preceded an incident, and who should be paged. The metadata needed to answer them is widely captured, so the interesting question is not whether it exists but whether what a system *represents* and what its interface can *ask* together suffice.

We treat a reliability workload as a requirements instrument. We specify a reference property graph for lakehouse operational metadata — eleven node labels, thirteen stored relationship labels spanning fifteen typed endpoint signatures, and one derived dataset-dependency relation, with schema and lineage edges carrying both valid time and system time — and define seven reliability queries formally, each reduced to the graph operations it needs. We validate the model by implementing it on an embedded property-graph engine over a synthetic lakehouse and executing all seven queries against expected results fixed in advance. We then assess six metadata systems under a stated rubric that separates representation from query expressiveness, deriving answerability from the two layers rather than judging it directly.

All seven queries execute and return the expected results. Among the six systems examined none answered more than three of the seven, and the binding constraint differed by system: Apache Atlas is limited by a query interface with no traversal construct despite storing its graph in a graph database, whereas Unity Catalog can express every required operation in recursive SQL but lacks the metadata to feed it. Per-hop latency, which freshness propagation requires, was represented by none. Separating the two layers is what makes these gaps actionable.

Keywords: Property graphs, Data lineage, Metadata management, Lakehouse, Temporal graphs, Data reliability

1 Introduction

A pipeline fails at 02:14. The engineer paged has perhaps ten minutes before a downstream pricing service serves stale numbers, and must answer four questions. Which consumers are affected, and how badly? What changed upstream shortly before the failure? Is the table stale because of this failure,

or because something three hops up has been late all night? Who owns the component to be fixed?

Each is a question about paths through a structure: a ranked reachability query; a temporal question, and a subtle one, since it asks what the pipeline *believed* when it ran; an aggregation along a path rather than over a set; and a path selection with a predicate on the endpoint.

The data is largely there. A modern lakehouse [5, 6] is surrounded by systems recording catalog entries, table and column lineage, schema history, quality assertions, ownership and freshness commitments. The research gap is therefore not absent metadata. It is that answering an operational question requires two things at once — that the system *represent* the relevant facts, and that the interface it *exposes* be able to state the required operation over them — and that these are routinely conflated when metadata systems are compared. A system can hold a complete lineage graph and offer no way to aggregate along it; another can offer a general query language over metadata that omits the quantity the question needs.

Research questions.

- RQ1** What metadata structure suffices to express common lakehouse reliability investigations, and is every element of it required by some investigation?
- RQ2** What graph-query capabilities do they require?
- RQ3** To what extent do contemporary metadata systems expose those capabilities, and is the binding constraint representation or expressiveness?
- RQ4** Can the reference model express the workload in an executable graph implementation?

Contributions.

A **reference property-graph model** (Sect. 3): eleven node labels, thirteen stored relationship labels spanning fifteen typed endpoint signatures, and one derived dataset-dependency relation, with schema and lineage edges carrying valid time *and* system time. A **formally specified workload** of seven reliability queries (Sect. 4), each with input, output and required operations, which converts “does this system have lineage?” into a question with a checkable answer. An **executable validation** (Sect. 6) on an embedded property-graph engine, all seven queries checked against expectations fixed beforehand. And a **reproducible assessment** of six systems under a stated rubric that separates the two layers and *derives* answerability from them.

The novelty is not that metadata can be modelled as a graph — Atlas stores its metadata in a graph database, and DataHub and Egeria expose

graph models — nor that column-level lineage is useful. It is the use of a reliability workload as a *requirements instrument*, and the layer separation that instrument exposes.

2 Background and Related Work

Provenance.

Buneman et al. [8] separated why- from where-provenance; Cheney et al. [9] and Herschel et al. [18] surveyed the field. Provenance semirings [15], extended to aggregation by Amsterdamer et al. [2], give an algebraic account of how annotations combine along derivation paths; freshness propagation (Q4) is an instance, with + along a path and max across paths. We differ in direction of travel: that literature is general and derives properties from general models, whereas we fix a few operational questions and ask what minimum structure answers them.

Metadata catalogs, and how they are compared.

Goods [16] catalogued Google’s datasets post hoc and found metadata must largely be inferred rather than declared; Ground [17] argued for a data context service spanning application, behaviour and change. Neither offers a workload against which a model can be judged. Comparisons of the deployed systems, meanwhile, are typically feature inventories — *has lineage*, *has column-level lineage*, *has a catalog* — which every candidate passes, and which therefore do not discriminate between a system that records a fact and one that can also answer a question about it. We found no published systematic comparison of these six against a stated reliability workload; that is an absence we did not fill by searching, not a proof that none exists.

Graph query languages and temporal databases.

Property graphs and their languages are well characterised [3, 14], with GQL and SQL/PGQ converged on a common pattern-matching core [13]. The operations our workload needs are ordinary there, so where a metadata system cannot express them that reflects its interface, not the state of the art. Valid and transaction time are long

established, standardised as application-time and system-time periods in SQL:2011 [22], and temporal property graphs are active [12] with implementations such as GRADOOP [27]. Our use is narrow and, as far as we can determine, not standard in operational metadata: both dimensions on lineage and schema *edges*.

Schema evolution, data quality, knowledge graphs.

Schema evolution in schema-flexible stores is well studied [20, 29], including migration cost at scale [21]; the lakehouse inherits it where consumers are pipelines. Deequ [28] and TFX data validation [7] established declarative assertions, with the management challenges set out by Polyzotis et al. [26]; we make assertions graph citizens so *coverage* becomes a reachability question. The knowledge-graph literature [1, 19] addresses heterogeneity and quality over graph data, and operational metadata is a small, well-typed instance — though Q7’s reading of a missing assertion is closed-world.

What this is not.

Not a lineage-visualisation or data-observability proposal: those render a graph an operator reads, whereas our concern is which operations must be *expressible* over it — which is why a good lineage explorer can still leave Q4 unanswerable. Nor a new metadata system.

3 A Reference Model for Operational Metadata

We model operational metadata as a directed, labelled property graph $G = (V, E, \lambda, \pi, \tau)$ where λ assigns labels, π assigns properties in the usual sense [3], and τ assigns to each temporal edge a pair of intervals defined in Sect. 3.1. Table 1 gives the edges and Fig. 1 shows the whole. The justification for each element is the query it serves; the released artefact tabulates that mapping in full, and no element is present that serves no query.

Nodes.

Eleven labels, each carrying only the properties the workload reads: Dataset(name), SchemaVersion(version_id, ordinal), Field(fqn, data_type, nullable),

Table 1 Stored relationship signatures of the reference model, with node labels abbreviated: Dataset, SV SchemaVersion, Field, Pipeline, Run, QA QualityAssertion, AR AssertionResult, Consumer, Principal, Incident. Thirteen relationship labels span fifteen signatures: ASSERTS_ON and OWNS each admit two endpoint pairs and are listed once per pair. † marks the two bitemporal relationships, which carry valid_from, valid_to, recorded_from and recorded_to. FEEDS is a *derived* dependency relation and is not stored (Sect. 3.2). Every signature here appears in Fig. 1, and none is absent from it.

Edge	From → To	Properties
READS, WRITES	P → D	bitemporal
HAS_SCHEMA†	D → SV	
DECLARES	SV → F	
DERIVES_FROM†	F → F	
INSTANCE_OF	R → P	dependency, subtype, bitemp.
PRODUCED	R → AR	
EVALUATES	AR → QA	
ASSERTS_ON	QA → D	
ASSERTS_ON	QA → F	
CONSUMES	C → D	
OWNS	Pr → D	
OWNS	Pr → P	
GOVERNED_BY	D → SLA	
AFFECTS	I → D	
FEEDS (<i>deriv.</i>)	D → D	latency_minutes

Pipeline(name, schedule_minutes), Run(run_id, scheduled_for, ended_at, status, delay_minutes), QualityAssertion(assertion_id, kind, severity), AssertionResult(result_id, outcome, observed_at), Consumer(name, criticality), Principal(name, kind), SLA(sla_id, max_staleness_minutes) and Incident(incident_id, detected_at).

Design decisions.

Three labels are less usual in a catalog. SchemaVersion is a node rather than a property, so a schema can be referred to, versioned and connected to the fields it declares; field membership is consequently schema-version-dependent, reached by Dataset HAS_SCHEMA SchemaVersion DECLARES Field, which is what lets Q2 ask about a field a *past* version declared. Run is separated from Pipeline, distinguishing a definition from its executions, without which root-cause questions cannot be posed. Consumer carries a criticality tier, so

impact is ranked rather than enumerated: an affected nightly report and an affected pricing service are not the same event. `AssertionResult` is separate from `QualityAssertion`, joined by `EVALUATES`, because Q7 asks whether an assertion *exists* and Q3 whether one *failed*; collapsing them leaves one unanswerable.

Column-level lineage is a first-class edge. `DERIVES_FROM` connects `Field` to `Field`, because dataset-level lineage over-approximates: it reports that a downstream table depends on an upstream one when the question is whether it depends on the column being dropped. We adopt OpenLineage’s *direct/indirect* distinction [24] in the **dependency** property: a column used in a filter or join affects an output without deriving it, and dropping a join key breaks a pipeline as surely as dropping a projected column.

3.1 Bitemporal schema and lineage

Both bitemporal edges carry two intervals, in the sense standardised for application and system time in SQL:2011 [22]:

Valid time [`valid_from`, `valid_to`] — the period during which the relationship held *in the operational world*.

System time [`recorded_from`, `recorded_to`] — the period during which the metadata system *held that belief*.

The two diverge whenever metadata arrives late or is corrected retrospectively — precisely the situation an incident investigation meets. In the running example a schema change on `orders_raw` became operationally valid at 01:50 but was first recorded at 02:10, so a pipeline running at 02:00 read data in the new shape while the catalog still answered with the old schema. At the 02:14 incident two questions have two answers:

valid at t : `valid_from` $\leq t <$ `valid_to` (1)

believed at b : `recorded_from` $\leq b <$ `recorded_to` (2)

With $t = 02:14$, (1) selects the new version; with $b = 02:00$, (2) selects the old one. A single version history, however carefully maintained, can represent one of these but not both — and the discrepancy is the explanation the engineer is

looking for. This is why we require both dimensions, and why Sect. 6 asks of each system whether it represents them.

3.2 The derived dependency relation

Q3, Q4, Q5 and Q7 traverse dataset-to-dataset dependency, which the model does *not* store as a primitive. Storing it would duplicate `READS` and `WRITES` and could drift from them. We define it instead as a derived relation:

$$d_1 \xrightarrow{\text{feeds}} d_2 \iff \exists p \in \text{Pipeline} : \quad (3)$$

$$p \xrightarrow{\text{reads}} d_1 \wedge p \xrightarrow{\text{writes}} d_2$$

and attribute to each hop a latency. Of the available readings — expected processing time, observed run delay, SLA lag, dataset freshness — we commit to one:

$$\text{latency}(e) := \text{delay}(r_{p(e)}^*) \quad (4)$$

$$\text{delay}(r) := r.\text{ended_at} - r.\text{scheduled_for} \quad (5)$$

for a hop e produced by pipeline $p(e)$, with r_p^* the most recent `SUCCESSFUL Run` of p . Hop latency is thus the *observed delay, against schedule, of the run that materialised the downstream dataset* — observable, already in the model via `Run`, and what an operator means by how much lateness a hop contributes.

That it must be computed before it can be traversed is part of what makes Q4 demanding in practice, and Sect. 6.3 reports that none of the systems examined represents an equivalent quantity.

Running example.

A synthetic retail lakehouse: `orders_raw` and `fx_rates` are ingested hourly; `fx_rates` is smoothed into `fx_rates_smoothed`; `clean_orders` reads `orders_raw` and `fx_rates_smoothed` to write `orders_clean`; and `agg_revenue` writes `daily_revenue`, consumed by a pricing service (criticality 1) and a finance dashboard (criticality 3). The example is synthetic throughout; no operational data from any organisation appears in this paper.

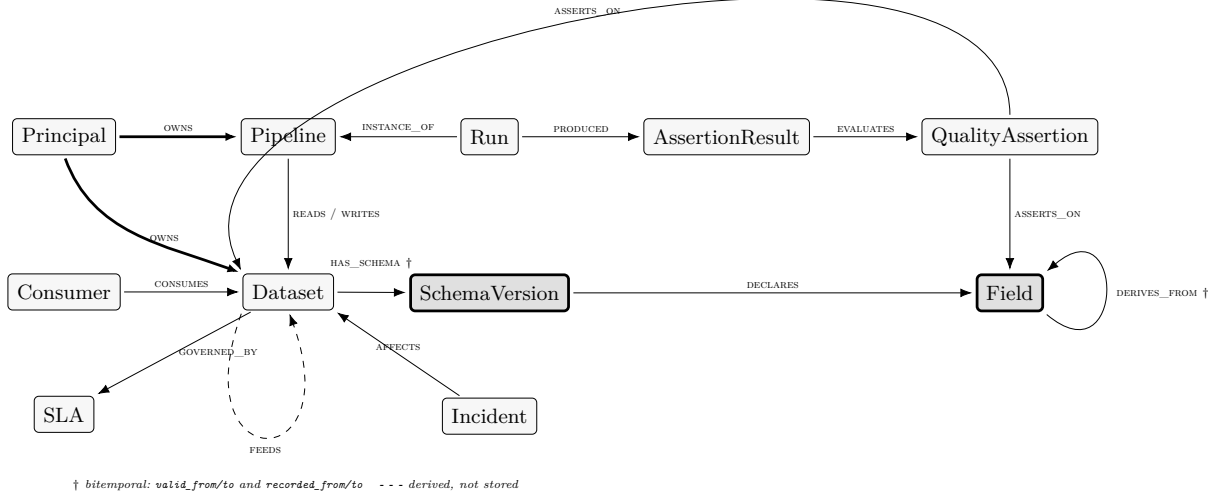


Fig. 1 The reference model. Shaded nodes are first-class here and usually attributes in a catalog. Solid edges are stored; the **dashed** self-loop on **Dataset** is the *derived* FEEDS relation of Eq. (3). The two edges marked † are bitemporal, carrying valid time and system time rather than a single write timestamp. Ownership is drawn heavier because Q5’s answer depends on which owned node is nearer the incident, and OWNS attaches to both datasets and pipelines. All fifteen stored endpoint signatures of Table 1 appear here, and no others.

4 The Reliability Query Workload

The workload is this paper’s instrument, so it is specified rather than described. Each query below is given its input, its output and the graph operations it requires. The artefact maps each requirement onto the model elements that satisfy it, which is the traceability supporting the minimality claim: an element serving no query is not in the model.

4.1 Workload semantics

Q1 Downstream impact.

In: dataset d_0 . *Out:* $(c, c.\text{criticality})$ for every Consumer c with $c \xrightarrow{\text{consumes}} d$ and $d_0 \xrightarrow{\text{feeds}^*} d$, ordered by criticality ascending. *Requires:* transitive closure over typed edges. Q1 is the control, keeping the assessment honest about what is genuinely hard.

Q2 Breaking change.

In: field f to be removed or retyped. *Out:* every field f' with $f' \xrightarrow{\text{derives_from}^+} f$, with hop distance. *Requires:* field-level lineage closure plus schema versioning to resolve f against the version that declared it. The discriminating case is INDIRECT dependency: a downstream field joining on **currency** without projecting it depends on it really, and a projection-only or dataset-level analysis calls the change safe.

Q3 Root-cause candidates.

In: incident i , window Δ . *Out:* upstream datasets u with $u \xrightarrow{\text{feeds}^*} d_0$, where $i \xrightarrow{\text{affects}} d_0$, such that some Run of a pipeline writing u either has **status** = FAILED or PRODUCED an **AssertionResult** with **outcome** = FAIL, and has **ended_at** $\in [i.\text{detected_at} - \Delta, i.\text{detected_at}]$. *Requires:* reverse traversal and a temporal predicate on run state. The second disjunct is why **AssertionResult** and **EVALUATES** exist: a run can succeed and still be the cause, and traversing **EVALUATES** to the **QualityAssertion** turns “some assertion failed” into a named one, which is the difference between a signal and an action.

Q4 Freshness propagation.

In: dataset d . *Out:* implied staleness, and whether it breaches the governing SLA. Let $\mathcal{P}(d)$ be the set of FEEDS paths ending at d . Then

$$L(P) = \sum_{e \in \text{hops}(P)} \text{latency}(e) \quad (6)$$

$$\text{ImpliedStaleness}(d) = \max_{P \in \mathcal{P}(d)} L(P)$$

with latency as committed in Eq. (4). *Requires:* three capabilities usually collapsed into “path aggregation” — variable-length traversal to enumerate $\mathcal{P}(d)$, a *path-local* fold scoped to one path to compute $L(P)$, and an aggregation *across* paths for the maximum. Reachability supplies neither of the last two: that d is reachable from a delayed source says nothing about how much delay arrives. The executed form is `evaluation/q4.cypher`; Kuzu has no `reduce`, so the path-local fold is written `list_sum(list_transform(rels(p), ...))`.

Q5 Escalation routing.

In: incident i with $i \xrightarrow{\text{affects}} d_0$. *Out:* one Principal to page, and the node it owns. Ownership attaches to *both* datasets and pipelines, so Q5 is defined over the alternating dependency path

$$d_0 \xleftarrow{\text{reads}} p_1 \xrightarrow{\text{writes}} d_1 \xleftarrow{\text{reads}} p_2 \xrightarrow{\text{writes}} \dots \quad (7)$$

rather than over FEEDS alone, which collapses p_k out of the path and would make a pipeline owner unreachable. Position interleaves the two kinds: d_k sits at $2k$ and p_{k+1} at $2k + 1$. Then: (i) criticality is a total order on integers, lower more critical; (ii) the target c^* is the reachable consumer of minimum criticality, ties broken by name; (iii) among nodes of a shortest dependency path from d_0 to c^* ’s dataset, take the one at minimum position carrying an `OWNS` edge; (iv) if several principals own it, the lexicographically first. *Requires:* path selection, and ownership resolvable in-graph over two node kinds.

This is the one query that needs the intermediate Pipeline, and it is why the `OWNS:Principal` $\rightarrow$ Pipeline signature is in the model at all. Q1, Q3, Q4 and Q7 are content with FEEDS; Q5 recovers the alternating path by expanding each FEEDS hop back through `via_pipeline`, which Eq. (3)

records, since no engine in common use offers variable-length traversal over a composite pattern.

Q6 Change attribution.

In: incident time t , belief time b . *Out:* for each SchemaVersion of the affected dataset, whether it satisfied (1) at t and whether it satisfied (2) at b . *Requires:* two independent temporal dimensions on one edge; where they disagree, the disagreement is the finding.

Q7 Coverage gaps.

In: criticality threshold k and an evaluation time t . *Out:* pairs (c, u) where $c.\text{criticality} \leq k$, $c \xrightarrow{\text{consumes}} d$, $u \xrightarrow{\text{feeds}^*} d$, and u is *unasserted* at t . The time is not optional: because `HAS_SCHEMA` is bitemporal, which fields a dataset declares is itself a function of when one asks, so coverage is judged against the version satisfying Eq. (1) at t . Four readings of “unasserted” are available — no assertion of any kind, none currently valid, none *passing*, or none covering the relevant field. We commit to the first, refined to the schema in force: u is unasserted at t iff no `QualityAssertion ASSERTS_ON u` and none `ASSERTS_ON` any Field declared by the schema version of u valid at t . Existence, not outcome: a *failing* assertion still counts as coverage, because Q7 asks what is unwatched and Q3 asks what is broken. *Requires:* negation *under* a transitive closure, with a temporal predicate selecting the schema version.

5 Assessment Method

Applying the workload to real systems requires stated criteria, or the resulting table is not reproducible. This section gives them.

5.1 Two layers

We assess on two independent axes, because conflating them hides the diagnosis. *Metadata representation* asks whether the system records the fact a query needs; *query expressiveness* asks whether the interface it exposes can state the required operation over what it records. A query is answerable only if both hold, so

$$\text{answerable}(Q, S) = \min \left(\min_{r \in \text{rep}(Q)} S_r, \min_{x \in \text{expr}(Q)} S_x \right) \quad (8)$$

over the order $N < P < Y$, with `rep` and `expr` the per-query requirements recorded in `capability_matrix.json`. Group (c) of Table 2 is computed by (8) rather than judged directly, so a reader who disputes a cell can locate the specific representation or expressiveness finding responsible for it.

5.2 Rubric

Y Native. The capability is available through a documented interface of the system as shipped, without reconstructing the graph outside it.

P Partial. Available only under a material qualification, which the cell’s reason states: connector-specific rather than a property of the core model, bounded, requiring a join to a separately-governed store, or semantically narrower than the workload needs.

N Absent. Not available through any documented or source-inspected interface we examined.

– *Not applicable.* The artefact defines no layer of this kind. `OpenLineage` specifies data and not a query surface, so grading it on expressiveness would be a category error.

5.3 Systems, versions and evidence

Six systems: the `OpenLineage` specification, Apache Atlas, DataHub, OpenMetadata, Egeria and Databricks Unity Catalog — five open source, one proprietary but openly documented. All assessments were made on 21 September 2026 against the most current documentation we could locate, and against source where documentation was silent on a point the workload depends on. Atlas was assessed against its `master` branch grammar rather than the 2.0.0 documentation still prominent in search results, because the DSL was rewritten after that release. The sources examined were each project’s published metadata model, query documentation and, where needed, source [4, 10, 11, 23–25]; one finding is version-dependent, since recursive CTEs require

Databricks Runtime 17.0 or later [10]. A per-system table of versions and the specific pages behind each column is in the artefact.

Every cell of Table 2 carries, in `evaluation/capability_matrix.json`, a verdict, a one-line reason and a citation key; the generator refuses to emit a cell lacking a source. The tables here are produced from that file, so no capability claim in this paper is hand-transcribed.

Two determinations rest on source rather than prose documentation, which we flag because it is unusual. Egeria’s `InstanceProperties` declares `effectiveFromTime` and `effectiveToTime` and `Relationship` carries `InstanceProperties`, so its edges are validity-dated [23] — which the documentation does not state. Conversely, Atlas’s `AtlasClassification` declares `validityPeriods` while `AtlasRelationship` declares only `createTime`, `updateTime` and `version`: valid time exists in Atlas, but on tags rather than on the edges a lineage query walks [4].

6 Evaluation

6.1 Reference implementation

We implemented the model on KUZU 0.11.3, an embedded property-graph engine with a Cypher dialect, chosen because it runs in-process and makes the artefact reproducible from one package install. Kuzu requires typed node and relationship tables, so the schema file is a machine-checked statement of the model: a query naming an undeclared edge fails at parse time rather than silently returning nothing. That is not incidental — an undefined `FEEDS` edge survived an earlier draft because nothing checked it.

The graph holds **45 nodes across 11 labels and 64 edges across 14 types**, 4 of them derived `FEEDS` edges materialised by Eq. (3). It is deliberately small, so every expected result is derivable by hand. It models the incident of Sect. 3.1 — a failed ingest at 01:45, a schema change valid at 01:50 and recorded at 02:10, a failing assertion at 02:02, an incident at 02:14 — with one unasserted upstream dataset and four distinct `FEEDS` paths into `daily_revenue`.

Porting notes: Kuzu has no `reduce`, so the path-local fold is `list_sum(list_transform(...))`, and an `UNWIND` node cannot be rebound as a pattern,

so Q5 resolves ownership by primary key. Neither changes the operations required.

6.2 Query validation (RQ4)

Each query was executed and compared with an expected result written down *before* the run. **All seven executed and all seven matched**; per-query capabilities, row counts and interpretations are in `evaluation/results.json`. This answers RQ4 affirmatively and, with the traceability mapping, RQ2 and the sufficiency half of RQ1: the model expresses the workload, and the operations it needs are the six in group (b) of Table 2. On the other half, necessity is *exhibited* rather than proved — every element is justified by a query that uses it, but we do not show no smaller model exists (Sect. 6.4).

Three results are worth reading rather than counting. Q4 instantiates Eq. (6) and returns an implied staleness of **36 minutes against a 30-minute SLA**, from `fx_rates` → `fx_rates_smoothed` → `orders_clean` → `daily_revenue` accumulating 20 + 12 + 4. The other three paths into `daily_revenue` total 16, 16 and 4, so neither reachability nor a per-dataset freshness value surfaces the breach — only the path-local sum followed by the cross-path maximum does. The 4-minute single-hop path makes the point sharply: a dataset one hop from the target can look timely while the graph behind it is half an hour late. Q6 returns both schema versions with opposite verdicts — v2 valid at the incident but not believed at 02:00, v1 believed then but no longer valid. Q7, evaluated at 02:14, returns one pair: the tier-1 pricing service and `fx_rates`, three hops upstream and unasserted under the schema valid at that instant. Q5 returns a *pipeline* owner: the incident dataset `orders_clean` is unowned, so the nearest owned node on the dependency path is `agg_revenue` at position 1, owned by the on-call rotation — nearer than `daily_revenue`’s own owner at position 2. Traversing only FEEDS would have collapsed that pipeline out of the path and paged the wrong team.

What this does *not* show: no timings were taken and none should be inferred. The graph is orders of magnitude smaller than a real catalog, and a capability expressible here may be impractical at scale.

6.3 Capability assessment (RQ3)

Table 2 gives both assessed layers and, in its third group, the answerability derived from them by Eq. (8).

Field-level lineage is comparatively mature.

Four of six represent it natively, the specification most richly: OpenLineage’s `columnLineage` facet distinguishes DIRECT from INDIRECT with subtypes down to JOIN, FILTER and WINDOW [24]. DataHub carries `fineGrainedLineages` inside `upstreamLineage` [11], OpenMetadata’s edges carry `columnsLineage` [25], and Egeria’s `DataMapping` maps a schema attribute of one asset to another [23]. Atlas scores P because `ColumnLineageProcess` arrives through the Hive bridge rather than the core type system [4]; Unity Catalog scores P for a documented reason bearing on Q2 — “*Lineage is not preserved for renamed catalogs, schemas, tables, views, or columns*” [10] — since a rename is exactly the change the query concerns.

The binding constraint differs by system, which is the main finding.

Atlas is limited by *expressiveness*: its current DSL grammar contains no traversal production at all — the `loop` and `withPath` clauses of the pre-1.0 language are absent from both lexer and parser grammar on `master`, the surviving aggregates apply to a result set with no path context, and Gremlin executes DSL internally but is not exposed [4]. Atlas stores its metadata in a graph database and still cannot express Q1, which is why the useful distinction is the interface rather than the storage engine.

Unity Catalog is the mirror image, limited by *representation*. Databricks SQL supports `WITH RECURSIVE` from Databricks Runtime 17.0, with a default limit of 100 recursion levels and a one-million-row bound [10], so a recursive CTE over `system.access.table_lineage` expresses each operation in group (b): closure and reverse traversal by choice of join direction, a path-local fold because a recursive CTE carries columns forward on each iteration, path selection because recursion depth is an ordinary column, temporal predicates as ordinary SQL inside the recursive step, and negation because `NOT EXISTS` can be applied to the materialised result. It has the strongest query

Table 2 Capability assessment. Y native, P partial, N absent, – not applicable (OpenLineage specifies data, not a query surface). A query needs its facts from (a) *and* its operations from (b); group (c) is *computed* from the two above it by Eq. (8), not assessed directly, so a disputed verdict there traces to a specific cell above. Groups (a) and (b) are generated from `evaluation/capability_matrix.json`, where every cell carries a reason and a source.

	OpenL.	Atlas	DataHub	OpenM.	Egeria	UC
(a) Metadata representation – <i>does the system record it?</i>						
dataset lineage	Y	Y	Y	Y	Y	Y
field-level lineage	Y	P	Y	Y	Y	P
schema versions	Y	P	Y	Y	Y	N
run history	Y	P	Y	Y	P	P
quality assertions	Y	N	Y	Y	P	P
ownership	N	Y	Y	Y	Y	Y
per-hop latency	N	N	N	N	N	N
valid time	N	P	N	N	Y	N
transaction time	P	Y	Y	Y	Y	P
(b) Query expressiveness – <i>can its interface ask?</i>						
transitive closure	–	N	Y	P	Y	Y
reverse traversal	–	N	Y	P	Y	Y
path-local aggregation	–	N	N	N	N	Y
path selection	–	N	N	N	P	Y
temporal predicates	–	P	Y	P	Y	Y
negation under closure	–	N	N	P	N	Y
(c) Workload answerability – <i>derived from (a) and (b) by Eq. (8)</i>						
Q1 Downstream impact	–	N	Y	P	Y	Y
Q2 Breaking change	–	N	Y	P	Y	N
Q3 Root-cause candidates	–	N	Y	P	P	P
Q4 Freshness propagation	–	N	N	N	N	N
Q5 Escalation routing	–	N	N	N	P	Y
Q6 Change attribution	–	P	N	N	Y	N
Q7 Coverage gaps	–	N	N	P	N	P

surface in the set; what it lacks is what to point it at.

Its data-quality position is narrower than it first appears, and we revised it during this pass. The system table `data_quality_monitoring.table_results` records freshness and completeness *results* per table, which corresponds to `AssertionResult`. There is no companion table of check *definitions*: anomaly detection is inferred from a table’s own history rather than authored, enabled per schema or catalog rather than declared per check, and table-scoped, with no column-level checks. Nothing corresponds to `QualityAssertion`, so the cell is P [10] — and that single change moves Q3 and Q7 from N to P by Eq. (8). We did not touch those verdicts; the derivation did.

On run history our earlier wording was too strong. No join between `job_run_timeline` and the lineage tables is documented; the documented joins are to `lakeflow.jobs`, `billing.usage` and `compute.clusters` on `workspace_id`, `job_id` and `run_id`. But a table-to-job association exists elsewhere: `upstream_jobs` carries `job_id` and

`workspace_id` beside the table identity, which are exactly those join keys. The cell stays P, now because that path needs anomaly detection enabled and covers only monitored tables — not because no mapping exists [10].

Per-hop latency is represented by none of the six, and that is why Q4 fails everywhere.

The Q4 row of Table 2(c) is uniformly N, and the derivation locates the cause as a representation gap, not a query gap. DataHub comes closest with a `FRESHNESS` assertion type [11], and Unity Catalog reports commit freshness as predicted versus actual commit time [10] — but both are per-table staleness signals, not a latency attributed to a dependency edge, so neither accumulates along a path. For Unity Catalog the query would be expressible the moment the quantity existed. We had supposed every ingredient for Q4 was stored somewhere in the set; separating the layers showed otherwise, and we record the correction rather than dropping the claim.

Temporal semantics are uneven.

Egeria alone satisfied our bitemporal criterion, on both nodes and edges [23]. DataHub and Open-Metadata record system time well — versioned aspects, and `major.minor` entity versions incrementing the major on column deletion [25] — but not valid time, so they answer “when did the metadata change” rather than “what was in force”. Unity Catalog’s lineage rows carry `event_time` under a rolling one-year retention window [10].

None answered more than three of the seven, and three answered none.

DataHub answers Q1–Q3, Egeria Q1, Q2 and Q6, and Unity Catalog Q1 and Q5, with Q3 and Q7 partial. The remaining three answer no query outright, though not for the same reason: Open-Metadata reaches P on four of the seven, more partials than any other system, and fails them on the query layer rather than the data; Atlas reaches P on one; and OpenLineage, having no query layer, is not gradeable here at all. We draw no overall ranking — Egeria’s three Y and two P are at least as strong a showing as DataHub’s three Y and none — because the point is not which system wins but that the required capabilities are distributed across systems rather than concentrated in any one.

6.4 Threats to validity

Construct validity.

The workload is seven queries from practitioner experience, not sampled from a measured incident corpus; a different seven would produce a different table. We expect the Q4 row to be robust, since it depends on a quantity none of the systems represents, but that is an expectation, not a result. Our formalisations also fix choices reasonable practitioners would make differently — one reading of latency out of four (Eq. 4), one of four readings of “unasserted”, one tie-breaking rule in Q5 — each stated where it is made so a reader can substitute their own.

Internal validity.

The implementation shows the model can express the workload; it does not establish minimality, only that a query exists for every element.

Expected results were fixed before execution, but written by the same author as the queries.

External validity.

Synthetic validation at 45 nodes says nothing about production behaviour, and there is no performance evaluation here at all. A capability marked Y may be prohibitively expensive on a real catalog, and one marked N may be irrelevant to workloads other than this one.

Product-version drift.

Five systems are actively developed and the sixth is a specification under revision. Assessments are dated 21 September 2026 and cite the artefact they rest on, but some will age: the recursive-CTE finding is itself a capability that post-dates Databricks Runtime 17.0. Our first pass cited Atlas 2.0.0 documentation, which predates the DSL rewrite; re-checking against `master` changed the conclusion, and the same risk applies to cells not re-checked as recently.

Source limitations.

We assess documented and source-inspected interfaces. A system may have undocumented internal capabilities, and a proprietary system may expose more to its own operators than its public documentation shows. Absence of evidence in documentation is reported as absence of a *documented* capability, which is the claim the rubric makes.

7 Discussion

Four implications follow from the evidence, and one caution.

Representation and expressiveness should be assessed separately.

This is the methodological point, and it changed our own conclusions. On a single axis Atlas and Unity Catalog both look like partial performers; on two axes they are opposite failures with opposite remedies — Atlas needs a query surface over a graph it already stores, Unity Catalog needs metadata to point an already-capable surface at. Advice not distinguishing these is not actionable.

The gap is often narrower than it appears.

For Unity Catalog, four of the five unanswerable queries would become answerable if the facts were recorded, the traversal machinery being already present. For the systems limited by expressiveness, GQL and SQL/PGQ standardised exactly the operations needed [13], and the semiring account of aggregation along paths is nearly two decades old [2, 15]. Little here awaits invention.

A thin query layer may be enough.

We are *not* arguing that metadata platforms should be replaced by native graph databases: Atlas already is one internally and scores worst here, which is the cleanest evidence that the storage engine is not the lever. A metadata system exposing its existing graph through a standard pattern-matching language, rather than a bespoke fixed-depth lineage endpoint, would gain Q1, Q4, Q5 and Q7 without changing how it stores anything.

Temporal modelling is where new representation is needed.

Valid time on lineage and schema edges cannot be retrofitted by a better interface. Egeria shows it is implementable within an open metadata standard, and since Egeria already consumes OpenLineage events, the components of a bitemporal lineage graph over lakehouse data exist in one ecosystem unassembled. Per-hop latency is the other missing quantity, and cheaper: the model derives it from run delay orchestrators already record.

A caution.

Nothing here establishes that these are the right seven queries, only that an organisation needing these answers must have a metadata stack that represents and can ask for them.

8 Limitations and Conclusion

Limitations.

Consolidating Sect. 6.4: the validation is synthetic and small; there is no latency or throughput measurement; the workload is seven queries from experience rather than a formal incident corpus; the sample is six systems, so proprietary platforms that may have solved parts of this privately

are unrepresented; the assessment interprets documentation and source at one date for rapidly evolving systems; and several formal choices are defensible rather than uniquely correct.

Conclusion.

Lakehouse reliability questions form a graph workload naturally, and writing them down formally exposes what a metadata layer must support: seven queries reduce to six graph operations over eleven node labels and fifteen stored endpoint signatures, and the reference model expresses all seven when executed. A compact workload is a usable requirements instrument, converting an unfalsifiable question about features into a checkable one about operations, and it is the part of this work most readily reused with a different set of queries. The practical gap in contemporary systems lies less between metadata captured and metadata missing than between metadata captured and metadata that can be *operationally queried*: among the six examined, the binding constraint was expressiveness for some and representation for others, and none answered more than three of the seven.

Future work and reproducibility.

A workload derived from a real incident corpus would let the queries be weighted by how often they are asked; beyond that, a scalability benchmark on catalog-sized graphs, incremental maintenance of the derived dependency relation under continuous ingestion, and integration with observability systems that already hold the run-delay data Q4 needs. Whether agents can be trusted to run such queries and act on the results during an incident is a further question this paper does not address. The reference model, synthetic graph, all seven queries, the validation runner and the capability matrix with per-cell evidence *will be made publicly available with the final version*; the capability table is generated from that data rather than transcribed.

References

- [1] Acosta M, Zaveri A, Simperl E, et al (2013) Crowdsourcing linked data quality assessment. In: ISWC

- [2] Amsterdamer Y, Deutch D, Tannen V (2011) Provenance for aggregate queries. In: PODS
- [3] Angles R, Arenas M, Barceló P, et al (2017) Foundations of modern query languages for graph databases. *ACM Comput Surv* 50(5)
- [4] Apache Software Foundation (2026) Apache atlas documentation and source. *Apache Atlas Type System*, <https://atlas.apache.org/2.0.0/TypeSystem.html>; *Apache Atlas DSL Advanced Search*, <https://atlas.apache.org/2.0.0/Search-Advanced.html>; *Apache Atlas Hook and Bridge for Apache Hive*, <https://atlas.apache.org/2.0.0/Hook-Hive.html>; *Apache Atlas Classification Propagation and Entity Audit*, <https://atlas.apache.org/2.0.0/ClassificationPropagation.html>; *Apache Atlas source: AtlasDSLParser.g4 and AtlasDSL Lexer.g4, master branch*, <https://github.com/apache/atlas>; *Apache Atlas source: AtlasRelationship and AtlasClassification*, <https://github.com/apache/atlas>. All accessed or inspected 21 September 2026
- [5] Armbrust M, Das T, Sun L, et al (2020) Delta lake: High-performance ACID table storage over cloud object stores. *Proc VLDB Endow* 13(12):3411–3424. <https://doi.org/10.14778/3415478.3415560>
- [6] Armbrust M, Ghodsi A, Xin R, et al (2021) Lakehouse: A new generation of open platforms that unify data warehousing and advanced analytics. In: CIDR
- [7] Breck E, Polyzotis N, Roy S, et al (2019) Data validation for machine learning. In: *MLSys*
- [8] Buneman P, Khanna S, Tan WC (2001) Why and where: A characterization of data provenance. In: *ICDT*
- [9] Cheney J, Chiticariu L, Tan WC (2009) Provenance in databases: Why, how, and where. *Found Trends Databases* 1(4)
- [10] Databricks (2026) Databricks and unity catalog documentation. *Unity Catalog Data Lineage*, <https://docs.databricks.com/aws/en/data-governance/unity-catalog/>
- data-lineage; Lineage System Tables Reference: system.access.table_lineage and system.access.column_lineage, https://docs.databricks.com/aws/en/admin/system-tables/lineage; Common Table Expressions: WITH RECURSIVE, https://docs.databricks.com/aws/en/sql/language-manual/sql-ref-syntax-qry-select-cte; Jobs System Tables Reference: system.lakeflow.job_run_timeline, https://docs.databricks.com/aws/en/admin/system-tables/jobs; INFORMATION_SCHEMA.TABLES, https://docs.databricks.com/aws/en/sql/language-manual/information-schema/tables; Data Quality Monitoring Results System Table: system.data_quality_monitoring.table_results, https://docs.databricks.com/aws/en/admin/system-tables/data-quality-monitoring; Anomaly Detection, https://docs.databricks.com/aws/en/data-governance/unity-catalog/data-quality-monitoring/anomaly-detection/.* All accessed or inspected 21 September 2026
- [11] DataHub Project (2026) Datahub documentation. *The DataHub Metadata Model*, <https://docs.datahub.com/docs/metadata-modeling/metadata-model>; *DataHub Assertion Entity*, <https://docs.datahub.com/docs/generated/metamodel/entities/assertion>; *DataHub GraphQL Queries: searchAcrossLineage*, <https://docs.datahub.com/docs/graphql/queries>. All accessed or inspected 21 September 2026
- [12] Debrouvier A, Parodi E, Perazzo M, et al (2021) A model and query language for temporal graph databases. *VLDB J* 30(5)
- [13] Deutsch A, Francis N, Green A, et al (2022) Graph pattern matching in GQL and SQL/PGQ. In: *SIGMOD*
- [14] Francis N, Green A, Guagliardo P, et al (2018) Cypher: An evolving query language for property graphs. In: *SIGMOD*, pp 1433–1445,

<https://doi.org/10.1145/3183713.3190657>

- [15] Green TJ, Karvounarakis G, Tannen V (2007) Provenance semirings. In: PODS
- [16] Halevy AY, Korn F, Noy NF, et al (2016) Goods: Organizing google’s datasets. In: SIGMOD, pp 795–806, <https://doi.org/10.1145/2882903.2903730>
- [17] Hellerstein JM, Sreekanti V, Gonzalez JE, et al (2017) Ground: A data context service. In: CIDR
- [18] Herschel M, Diestelkämper R, Ben Lahmar H (2017) A survey on provenance: What for? what form? what from? VLDB J 26(6)
- [19] Hogan A, Blomqvist E, Cochez M, et al (2021) Knowledge graphs. ACM Comput Surv 54(4)
- [20] Klettke M, Störl U, Scherzinger S (2015) Schema extraction and structural outlier detection for json-based NoSQL data stores. In: BTW
- [21] Klettke M, Störl U, Shenavai M, et al (2016) NoSQL schema evolution and big data migration at scale. In: IEEE Big-Data, pp 2764–2774, <https://doi.org/10.1109/BigData.2016.7840924>
- [22] Kulkarni KG, Michels J (2012) Temporal features in SQL:2011. SIGMOD Rec 41(3)
- [23] LF AI & Data Foundation (2026) Egeria documentation and source. *Egeria Lineage Concepts and Open Metadata Types*, <https://egeria-project.org/concepts/lineage/>; *Egeria Effectivity Dates*, <https://egeria-project.org/parameters/overview/>; *Egeria source: InstanceProperties*, <https://github.com/odpi/egeria>. All accessed or inspected 21 September 2026
- [24] OpenLineage Project (2026) Openlineage specification. *OpenLineage Object Model and Dataset Facets*, [https://openlineage.io/docs/spec/object-model](https://openlineage.io/docs/spec/object-model;); *Column Lineage Dataset Facet*, https://openlineage.io/docs/spec/facets/dataset-facets/column_lineage_facet. All accessed or inspected 21 September 2026
- [25] OpenMetadata Project (2026) Openmetadata documentation. *OpenMetadata Lineage Standard*, <https://openmetadatastandards.org/lineage/lineage/>; *OpenMetadata Metadata Versioning*, <https://docs.open-metadata.org/v1.12.x/connectors/ingestion/versioning>; *OpenMetadata Advanced Search*, <https://docs.open-metadata.org/latest/how-to-guides/data-discovery/advanced>. All accessed or inspected 21 September 2026
- [26] Polyzotis N, Roy S, Whang SE, et al (2017) Data management challenges in production machine learning. In: SIGMOD
- [27] Rost C, Gomez K, Täschner M, et al (2021) Distributed temporal graph analytics with GRADOOP. VLDB J 31(2):375–401. <https://doi.org/10.1007/s00778-021-00667-4>
- [28] Schelter S, Lange D, Schmidt P, et al (2018) Automating large-scale data quality verification. Proc VLDB Endow 11(12)
- [29] Scherzinger S, Klettke M, Störl U (2013) Managing schema evolution in NoSQL data stores. In: DBPL